

ACTIVE-IA

Devolución de Trabajo Práctico

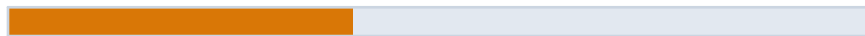
Materia:	PROG4A26 - Programacion 4 Agosto 2026
Comisión:	COMI-12 (2026)
Trabajo:	Trabajo Práctico - Fundamentos de Spring Boot
Alumno:	Miguel Angel Pichulman Cari

Calificación Final: 91/100

EVALUACIÓN POR CRITERIOS

C1: Desarrollo de Clases y Relaciones (UML)

6/15



Se observa una correcta definición de las entidades de dominio y el uso de herencia con la clase Base. Sin embargo, se detectaron omisiones importantes respecto al diseño UML: falta la interfaz 'Calculable', no se implementó su método 'calcularTotal()' en la clase 'Pedido' y tampoco se incluyeron los métodos específicos de negocio en 'Pedido' (addDetallePedido, findDetallePedidoByProducto, deleteDetallePedidoByProducto). Además, en la clase 'Base' se utilizó el atributo 'eliminatedAt' (LocalDate) en lugar de 'createdAt' (LocalDateTime) especificado en la rúbrica.

DESGLOSE POR SUBCRITERIO

C1.1 3/5 · WARNING

Las clases principales y ENUMs se crearon correctamente, pero falta la interfaz Calculable y la clase Base tiene 'eliminatedAt' en vez de 'createdAt'.

C1.2 0/5 · ERROR

No se implementó ninguno de los métodos requeridos en Pedido (addDetallePedido, findDetallePedidoByProducto, deleteDetallePedidoByProducto) ni el método calcularTotal() de la interfaz Calculable.

C1.3 3/5 · WARNING

La herencia y las relaciones estructurales uno-a-muchos (con List) están muy bien logradas. No obstante, falta la realización de Calculable por parte de Pedido.

C2: Configuración del proyecto Spring Initializr

10/10



El archivo de build 'build.gradle' está correctamente configurado con todas las dependencias requeridas para un entorno Spring Boot completo con base de datos H2 en memoria y Lombok.

DESGLOSE POR SUBCRITERIO

C2.0 2/2 · OK

Archivo build.gradle presente en la raíz del proyecto.

C2.1 2/2 · OK

Inclusión exitosa de la dependencia starter de web.

C2.2 2/2 · OK

Inclusión de spring-boot-starter-data-jpa.

C2.3 2/2 · OK

Inclusión de Lombok para agilizar el boiler-plate de entidades y DTOs.

C2.4 1/1 · OK

Inclusión de la dependencia del motor de base de datos H2.

C2.5 1/1 · OK

Inclusión de las DevTools para la recarga en desarrollo.

C3: Desarrollo de la capa de DTOs

30/30



Excelente organización. El alumno ha creado exactamente los 13 archivos DTO solicitados, organizados en subcarpetas lógicas y respetando la asimetría de diseño requerida por la consigna (Pedido sin Create y DetallePedido sin Edit).

DESGLOSE POR SUBCRITERIO

C3.1 7/7 · OK

CategoriaCreate, CategoriaDto y CategoriaEdit existen y están correctamente estructuradas.

C3.2 5/5 · OK

DetallePedidoCreate y DetallePedidoDto implementados correctamente sin edit innecesario.

C3.3 4/4 · OK

PedidoDto y PedidoEdit implementados correctamente respetando la asimetría planteada.

C3.4 7/7 · OK

ProductoCreate, ProductoDto y ProductoEdit creados de acuerdo a las especificaciones.

C3.5 7/7 · OK

UsuarioCreate, UsuarioDto y UsuarioEdit presentes con los atributos correctos.

C4: Instanciación de objetos a partir de DTOs

25/25



Se ha implementado un cargador de datos iniciales impecable a través de un DataLoader que utiliza CommandLineRunner. Se mapean y persisten de forma exacta las cantidades requeridas de cada entidad mediante sus respectivos DTOs.

DESGLOSE POR SUBCRITERIO

C4.0 5/5 · OK

DataLoader implementa CommandLineRunner y está decorado con @Component.

C4.1 4/4 · OK

Se instanciaron y guardaron exactamente 2 Usuarios.

C4.2 8/8 · OK

Se crearon 3 Pedidos utilizando listas de DetallePedidoCreate con cantidades de productos correctas.

C4.3 4/4 · OK

Se instanciaron 3 Categorías mediante CategoriaCreate.

C4.4 4/4 · OK

Se cargaron de forma exitosa los 10 Productos requeridos.

C5: Comprensión y aplicación de conceptos de Spring Boot (Buenas Prácticas)

20/20



Se aplican excelentes prácticas de desarrollo con Spring Boot. Se destaca el uso consistente de inyección de dependencias por constructor en lugar de la inyección directa en atributos, y una excelente modularidad de la aplicación.

DESGLOSE POR SUBCRITERIO

C5.0 6/6 · OK

La clase principal TrabajoPractico1Application está configurada adecuadamente como entrypoint.

C5.1 5/5 · OK

Inyección de dependencias implementada limpia y explícitamente mediante constructores en todos los servicios y en el DataLoader.

C5.2 5/5 · OK

Uso consistente y correcto de los estereotipos @Component, @Service y @Repository.

C5.3 4/4 · OK

El archivo application.properties contiene la configuración idónea para la consola H2 y el comportamiento de Hibernate.

FORTALEZAS

- Excelente estructuración y completitud de la capa de DTOs, respetando al detalle la asimetría de diseño sugerida.

- Impecable flujo de carga de datos iniciales en la clase DataLoader a través de los correspondientes servicios y DTOs de creación.
- Uso riguroso de buenas prácticas arquitectónicas, en especial la inyección de dependencias por constructor sin recurrir a anotaciones por campo (@Autowired).

RECOMENDACIONES

1. Crear la interfaz Calculable con el método calcularTotal(): void y hacer que la clase Pedido la implemente para cumplir con el diseño UML.
2. Implementar los métodos de negocio específicos de Pedido, como addDetallePedido(int, Producto), findDetallePedidoByProducto(Producto) y deleteDetallePedidoByProducto(Producto).
3. Asegurar la exactitud de los atributos heredados de Base (modificar 'eliminatedAt' de tipo LocalDate por 'createdAt' de tipo LocalDateTime).

COMENTARIOS DEL EVALUADOR

Miguel Angel, felicitaciones por la entrega. Has desarrollado una aplicación en Spring Boot sumamente estructurada, limpia y robusta, con un impecable manejo de DTOs e inyección de dependencias. Para lograr un trabajo perfecto, te sugerimos ajustar los detalles del modelo de clases referidos a la interfaz de cálculo y los métodos específicos de negocio en la entidad Pedido.